

Contents

1	Foundations	1
1.1	The Neuron as a Linear Model	2
1.1.1	Mathematical Formulation	2
1.1.2	Geometric Interpretation: The Hyperplane Decision Boundary	2
1.2	Introducing Non-Linearity: Activation Functions	3
1.2.1	Key Properties of Activation Functions	4
1.2.2	Common Activation Functions	5
1.3	Building Neural Networks	7
1.3.1	The Multi-Layer Perceptron (MLP)	7
1.3.2	Architecture and Forward Propagation	8
1.3.3	Expressive Power and the Universal Approximation Theorem	9
1.3.4	Deep vs. Shallow Networks	10
1.4	Learning as Optimization	11
1.4.1	Empirical Risk Minimization	11
1.4.2	Loss Functions	12
1.5	Algorithms for Gradient-Based Optimization	13
1.5.1	Gradient Descent	13
1.5.2	Optimization Variants	13
1.6	The Learning Engine: Backpropagation	14
1.6.1	Forward and Backward Propagation	15
1.7	Specialized Architectures	15
1.7.1	Convolutional Neural Networks (CNNs)	15
1.7.2	Recurrent Neural Networks (RNNs)	16
1.7.3	Autoencoders (AEs)	16
1.7.4	Transformer Networks	17
1.7.5	Core Architectural Components: Building Blocks of the Transformer	19
1.7.6	The Computational Bottleneck of Transformers	23
1.8	The Bigger Picture	23
	Appendices	28

List of Figures

1.1	Computational graph of a single artificial neuron. Inputs x_i are weighted by w_i and summed with bias b to produce z	2
1.2	Geometric interpretation in 2D space. The line $x_1 + 2x_2 - 3 = 0$ divides the plane. The weight vector $\mathbf{w}$ is normal to the decision boundary, pointing toward the region where $z > 0$	3
1.3	Comparison of Tanh and Sigmoid. Tanh is zero-centered with stronger gradients near zero.	6
1.4	Modern activation functions compared to ReLU. GELU and Swish introduce smoothness and non-monotonicity.	7
1.5	Architecture of a fully connected MLP with 4 inputs, two hidden layers (5 and 4 neurons), and 2 outputs. The orange path shows signal flow from x_2 to y_1 through weighted connections and activation functions.	9
1.6	A ReLU network with 3 hidden neurons creates 3 hyperplanes that partition the 2D input distinct regions.	10
1.7	Visualization of the Gradient Descent trajectory on a convex error surface.	13
1.8	Simple CNN architecture: input, convolution + activation, pooling, and fully connected layers Waseem et al. 2017.	15
1.9	RNN unrolled in time.	16
1.10	Autoencoder architecture: The input is compressed into a lower-dimensional latent space (Bottleneck) and then reconstructed.	17
1.11	Complete Encoder-Decoder Transformer architecture. The encoder output $\mathbf{H}_{\text{enc}}$ (highlighted in blue) is fed to all decoder layers for cross-attention. Residual connections (curved arrows) enable stable gradient flow throughout the network.	18
1.12	Transformer Encoder layer architecture. Residual connections (curved arrows on the left) bypass each sub-layer, ensuring stable gradient flow. Pre-layer normalization is applied before each transformation.	20
1.13	Transformer Decoder layer architecture. Note the three sub-layers: masked self-attention (to look at previously generated tokens), cross-attention (to read from encoder), and feed-forward network. Residual connections bypass each sub-layer.	21

List of Tables

1.1	Comparison of activation function properties.	7
1.2	Comparative summary of shallow and deep architectures.	11

Chapter 1

Foundations

Artificial Intelligence represents the broad ambition to create systems capable of autonomous reasoning and learning. Within this field, Deep Learning, a specialized subset, employs multi-layered neural networks to automatically learn complex representations without manual feature engineering. This capability has produced breakthrough results in computer vision and natural language processing, fueled by massive datasets and advances in computational power Goodfellow et al. 2016. Yet this success comes at a cost: state-of-the-art models now contain billions of parameters, creating critical deployment challenges. Understanding these architectures deeply is essential not only for building them, but for compressing them effectively which is the central challenge of this report.

This chapter establishes the theoretical foundation for these systems. We begin with the artificial neuron, examining its mathematical formulation and the role of non-linearity in capturing complex patterns. We construct the Multi-Layer Perceptron, analyze its architecture and information flow, and establish its expressive power through the universal approximation theorem Hornik et al. 1989. We formalize the optimization framework through loss functions and gradient descent, then explain backpropagation—the algorithm that computes gradients efficiently Rumelhart et al. 1986.

While recurrent architectures provided early solutions for sequence modeling, their sequential nature imposed significant limitations. We therefore examine the transformer, which achieves parallel computation through self-attention Vaswani et al. 2017. We dissect its encoder-decoder architecture and attention mechanism, building intuition through the Query-Key-Value framework. We analyze multi-head attention’s ability to capture diverse relationships and conclude by examining the quadratic complexity bottleneck—a key target for the compression techniques developed in subsequent chapters.

1.1 The Neuron as a Linear Model

1.1.1 Mathematical Formulation

The artificial neuron, formally introduced by McCulloch et al. 1943, serves as the fundamental building block of neural networks. Mathematically, it performs an **affine transformation** on its inputs followed by a non-linear activation.

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (1.1)$$

Given an input $\mathbf{x} \in \mathbb{R}^n$, weights $\mathbf{w} \in \mathbb{R}^n$ scale the importance of each feature, while the bias $b \in \mathbb{R}$ acts as an offset. This bias allows the decision boundary to shift independent of input values Goodfellow et al. 2016. The resulting logit z is subsequently passed through a non-linear activation function, $a = \phi(z)$.

The term $\mathbf{w}^T \mathbf{x}$ measures the alignment between the input and weight vectors. A large magnitude indicates a strong correlation (or anti-correlation) between the input pattern and the feature the neuron is tuned to detect Haykin 1999.

Diagram: Single Neuron with Affine Transformation

Figure 1.1 illustrates the computational flow: inputs are weighted, summed with the bias, and aggregated into the pre-activation z .

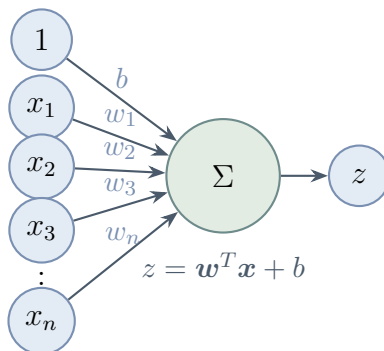


Figure 1.1: Computational graph of a single artificial neuron. Inputs x_i are weighted by w_i and summed with bias b to produce z .

1.1.2 Geometric Interpretation: The Hyperplane Decision Boundary

The equation $\mathbf{w}^T \mathbf{x} + b = 0$ defines a hyperplane partitioning the space $\mathbb{R}^n$ into two regions. This geometric view, central to the Perceptron Rosenblatt 1958, highlights the roles of the parameters:

- **Weight Vector $\mathbf{w}$:** Defines the orientation of the hyperplane. It is orthogonal to the decision surface.
- **Bias b :** Determines the distance of the hyperplane from the origin, given by $-b/\|\mathbf{w}\|_2$.

The pre-activation z is proportional to the signed distance from $\mathbf{x}$ to the hyperplane. Consequently, a single neuron functions as a binary linear classifier, capable of solving linearly separable problems (e.g., AND, OR) but failing on non-linear tasks such as XOR (Minsky et al. 1969).

Example: Geometric Interpretation in 2D

Consider an example in $\mathbb{R}^2$ with weights $\mathbf{w} = [1, 2]^T$ and bias $b = -3$. The decision boundary is the line where $z = 0$:

$$x_1 + 2x_2 - 3 = 0 \quad \Rightarrow \quad x_2 = \frac{3 - x_1}{2} \quad (1.2)$$

Figure 1.2 visualizes this boundary. The weight vector is normal to the line, pointing toward the positive half-space ($z > 0$).

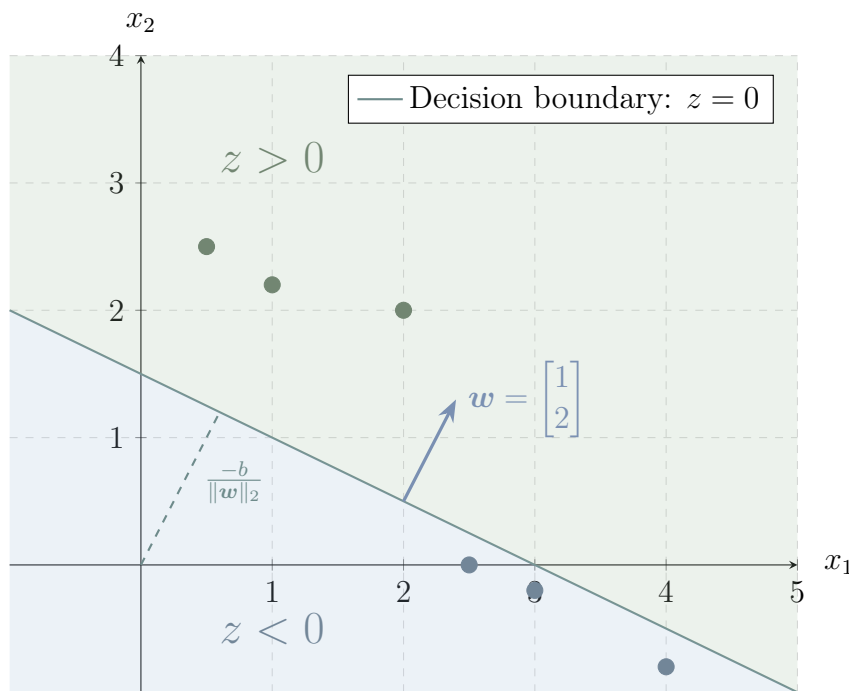


Figure 1.2: Geometric interpretation in 2D space. The line $x_1 + 2x_2 - 3 = 0$ divides the plane. The weight vector $\mathbf{w}$ is normal to the decision boundary, pointing toward the region where $z > 0$.

1.2 Introducing Non-Linearity: Activation Functions

Neural networks model complex, non-linear relationships. Without non-linearity, they collapse to simple linear transformations.

Why Non-Linearity Matters: A two-layer network without activation functions:

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{\mathbf{W}_2\mathbf{W}_1}_{\mathbf{W}'}\mathbf{x} + \underbrace{\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2}_{\mathbf{b}'} \quad (1.3)$$

This is equivalent to a single linear layer $\mathbf{y} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$. Without activation functions, depth is meaningless that implies that the mapping remains linear.

Definition 1.1 (Activation Function)

An **activation function** $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a non-linear function applied element-wise:

$$\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}) = \sigma(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}) \quad (1.4)$$

where $\mathbf{z}^{(\ell)}$ is the pre-activation and $\mathbf{a}^{(\ell)}$ the post-activation of layer ℓ .

Activation functions introduce non-linearity and control gradient flow during backpropagation.

1.2.1 Key Properties of Activation Functions

Differentiability and Gradient Flow

Backpropagation relies on the chain rule:

$$\frac{\partial \mathcal{L}}{\partial z_i^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}} \cdot \sigma'(z_i^{(\ell)}) \quad (1.5)$$

Theorem 1.1 (Gradient Flow Through Layers)

The gradient with respect to layer ℓ parameters involves:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} \propto \prod_{k=\ell+1}^L \mathbf{W}^{(k)} \text{diag}(\sigma'(\mathbf{z}^{(k-1)})) \quad (1.6)$$

This product determines whether gradients vanish (product $\ll 1$) or explode (product $\gg 1$).

Saturation and Vanishing Gradients

Definition 1.2

An activation function σ **saturates** in region $\mathcal{R}$ if $\lim_{x \in \mathcal{R}} |\sigma'(x)| \rightarrow 0$.

The Vanishing Gradient Problem

When neurons saturate, $\sigma'(z) \approx 0$, causing:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} \approx \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} \cdot \prod_{k=\ell+1}^L \mathbf{W}^{(k)} \underbrace{\text{diag}(\sigma'(\mathbf{z}^{(k-1)}))}_{\approx 0} \quad (1.7)$$

Gradients in early layers become exponentially small, preventing learning in deep networks with saturating activations like sigmoid and tanh.

Zero-Centeredness

Definition 1.3

An activation function σ is **zero-centered** if $\sigma(0) = 0$ and $\sigma(-x) = -\sigma(x)$ (odd symmetry).

Sparsity

Definition 1.4

The **sparsity** of σ on distribution P_X is: $\text{Sparsity}(\sigma) = \Pr_{X \sim P_X}[\sigma(X) = 0]$

Sparse representations offer computational efficiency, memory compression, and interpretability.

1.2.2 Common Activation Functions

Activation functions introduce non-linearities into neural networks, allowing them to learn complex patterns. They are generally categorized into classical saturating functions and modern non-saturating rectified units.

Classical Functions: Sigmoid and Tanh

Historically used in early multilayer perceptrons, these functions are now primarily used in output layers (binary classification) or specific RNN gates.

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$. It maps input to $(0, 1)$, allowing a probabilistic interpretation. However, it suffers from vanishing gradients (max derivative is 0.25) and outputs are not zero-centered.
- **Tanh:** $\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}$. It maps input to $(-1, 1)$. While it is zero-centered (aiding convergence), it still suffers from saturation and vanishing gradients for large $|x|$.

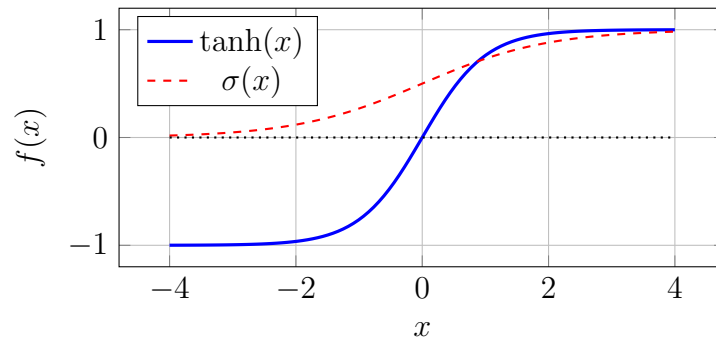


Figure 1.3: Comparison of Tanh and Sigmoid. Tanh is zero-centered with stronger gradients near zero.

Rectified Linear Units (ReLU) and Variants

ReLU defined as $f(x) = \max(0, x)$, became the standard for deep learning due to computational efficiency and the mitigation of the vanishing gradient problem for positive inputs. Its main drawback is the "Dying ReLU" problem, where neurons effectively die if weights update such that $wx + b < 0$ consistently.

To address this, several variants exist:

- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ with a small constant α (e.g., 0.01) to allow gradient flow for $x < 0$.
- **ELU (Exponential Linear Unit):** Uses $\alpha(e^x - 1)$ for $x \leq 0$. It offers smooth negative saturation and closer-to-zero mean activations but is computationally more expensive.

Modern State-of-the-Art: GELU and Swish

Recent transformer architectures (e.g., BERT, GPT) utilize smooth, non-monotonic activation functions.

- **GELU (Gaussian Error Linear Unit):** $x \cdot \Phi(x)$, where Φ is the standard normal CDF. It weighs inputs by their probability, preserving information more effectively than ReLU's hard threshold.
- **Swish/SiLU:** $f(x) = x \cdot \sigma(\beta x)$. It is self-gated, unbounded above, and bounded below, often outperforming ReLU in deeper networks.

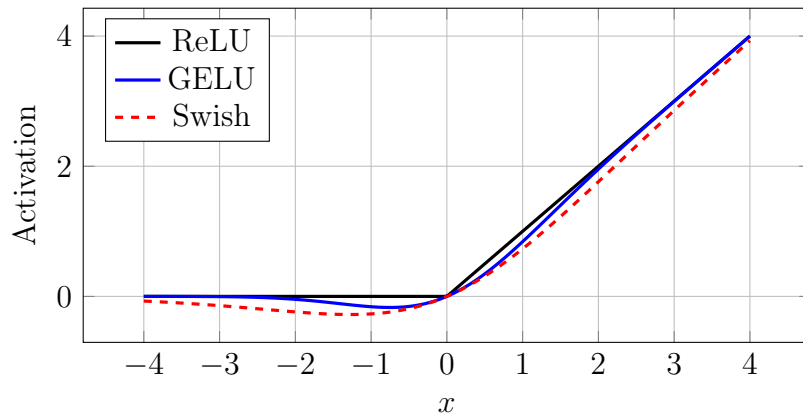


Figure 1.4: Modern activation functions compared to ReLU. GELU and Swish introduce smoothness and non-monotonicity.

Table 1.1: Comparison of activation function properties.

Activation	Range	Monotonic	Saturating	Zero-Centered	Sparse	Smooth
Sigmoid	$(0, 1)$	Yes	Both	No	No	Yes
Tanh	$(-1, 1)$	Yes	Both	Yes	No	Yes
ReLU	$[0, \infty)$	Yes	Left	No	Yes	No
Leaky ReLU	$(-\infty, \infty)$	Yes	No	No	No	No
ELU	$[-\alpha, \infty)$	Yes	Left	$\approx$ Yes	No	$\approx$ Yes
GELU	$(-\infty, \infty)$	No	No	Yes	No	Yes
Swish	$[-0.28, \infty)$	No	Left	$\approx$ Yes	No	Yes

1.3 Building Neural Networks

Neural networks are computational models inspired by the brain that extract patterns by propagating inputs through stacked layers of simple units. This section introduces the Multi-Layer Perceptron (MLP): its architecture, notation, and the core math behind its expressive power.

1.3.1 The Multi-Layer Perceptron (MLP)

Going from a single neuron to an MLP greatly increases expressiveness: a lone neuron can only separate data linearly, whereas multiple layers enable arbitrarily complex nonlinear mappings Rumelhart et al. 1986. MLPs are *feedforward* (no cycles) and typically *fully connected* (each neuron connects to all neurons in the next layer).

Definition 1.5 (Multi-Layer Perceptron)

An MLP is a feedforward network with

- input layer of dimension n_0 ,
- $L - 1$ hidden layers where layer ℓ has n_ℓ neurons ($\ell = 1, \dots, L - 1$),

- output layer of dimension n_L .

Each layer applies an affine map followed by a nonlinearity:

$$\mathbf{h}^{(\ell)} = \phi_\ell(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \mathbf{h}^{(0)} = \mathbf{x},$$

with $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$. The affine+nonlinear composition is what yields the network's representational power.

Diagram: Multi-Layer Perceptron Architecture

Figure 1.5 illustrates a typical MLP with one input layer, two hidden layers, and one output layer. Each neuron in a layer is connected to all neurons in the subsequent layer, forming a dense or fully connected architecture. The diagram highlights the flow of information for a single path and details the dimensions of the parameter tensors (weights and biases) at each stage.

1.3.2 Architecture and Forward Propagation

Forward propagation is the sequential transformation of input data into predictions. Conceptually, the initial data $\mathbf{x}$ enters the first layer and is transformed layer-by-layer until the final output $\hat{\mathbf{y}}$ is generated.

Definition 1.6 (Forward Propagation)

Given an input $\mathbf{x} = \mathbf{h}^{(0)} \in \mathbb{R}^{n_0}$ and parameters $\theta = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L$, the network computes activations recursively for $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)} \tag{1.8}$$

$$\mathbf{h}^{(\ell)} = \phi_\ell(\mathbf{z}^{(\ell)}) \tag{1.9}$$

The final prediction is $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$.

Matrix Representation (Batch Processing). In practice, training utilizes *vectorization* to process a mini-batch of m examples simultaneously, exploiting the parallel architecture of modern GPUs. If inputs are stacked as rows in a matrix $\mathbf{X} \in \mathbb{R}^{m \times n_0}$, the operations in Eq. 1.8–1.9 generalize to matrix operations:

$$\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}(\mathbf{W}^{(\ell)})^T + \mathbf{1}_m(\mathbf{b}^{(\ell)})^T, \quad \text{and} \quad \mathbf{H}^{(\ell)} = \phi_\ell(\mathbf{Z}^{(\ell)}), \tag{1.10}$$

where $\mathbf{1}_m$ is a column vector of ones (broadcasting the bias). This formulation replaces explicit loops with optimized linear algebra routines, significantly accelerating training stability and speed.

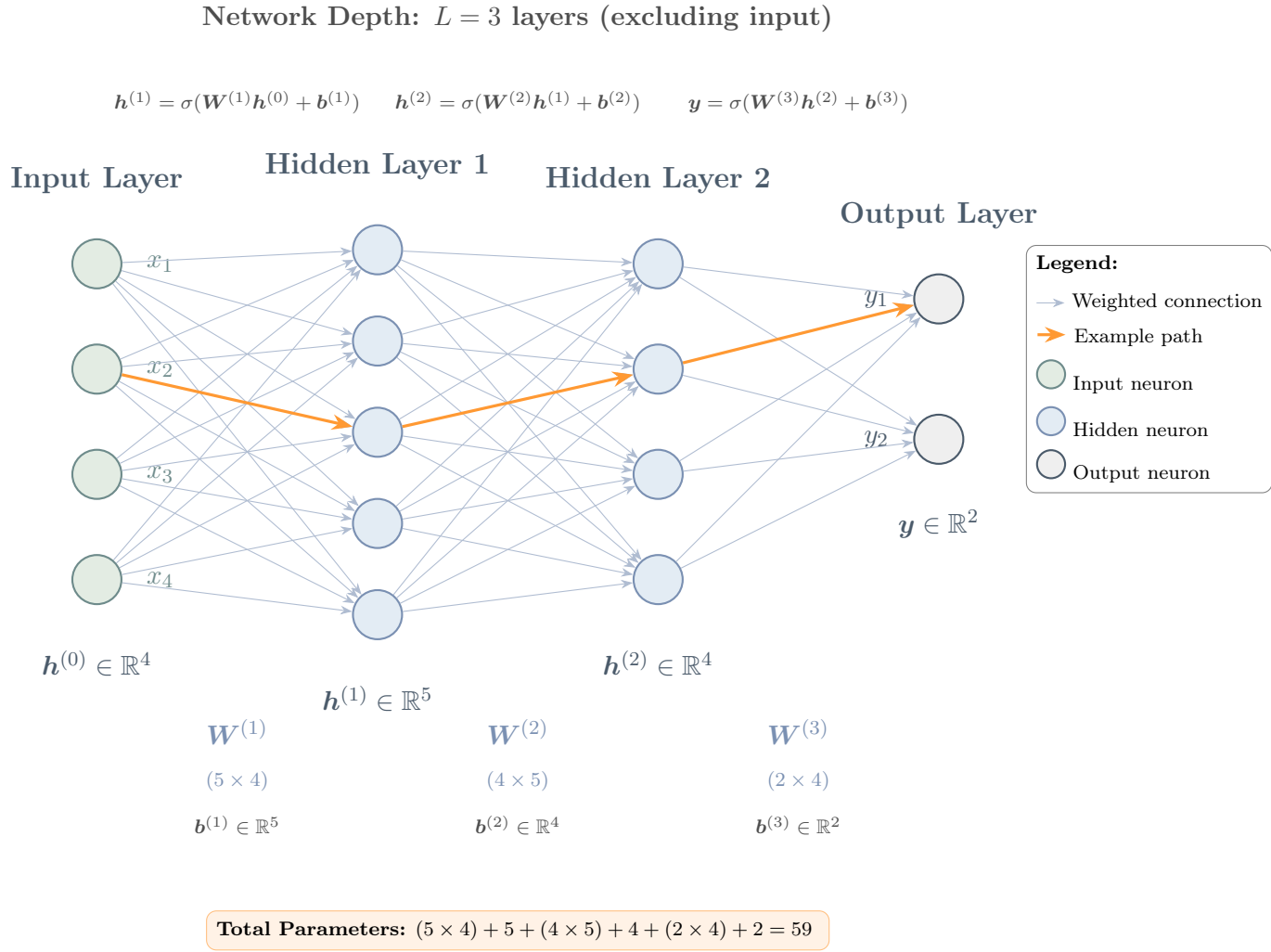


Figure 1.5: Architecture of a fully connected MLP with 4 inputs, two hidden layers (5 and 4 neurons), and 2 outputs. The orange path shows signal flow from x_2 to y_1 through weighted connections and activation functions.

1.3.3 Expressive Power and the Universal Approximation Theorem

The theoretical foundation of neural networks lies in the *Universal Approximation Theorem*, which establishes that shallow networks are dense in the space of continuous functions. In essence, a single hidden layer is sufficient to approximate any continuous function to arbitrary precision, provided the network is sufficiently wide.

Theorem 1.2 (Universal Approximation Theorem — Cybenko, 1989 Cybenko 1989)
Let ϕ be a non-constant, bounded, and continuous activation function. For any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ and error tolerance $\epsilon > 0$, there exists a network

with N hidden neurons,

$$F(\mathbf{x}) = \sum_{j=1}^N \alpha_j \phi(\mathbf{w}_j^T \mathbf{x} + b_j), \quad (1.11)$$

such that $|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon$ for all $\mathbf{x} \in [0, 1]^n$.

Geometric Intuition: Hyperplane Arrangements. To understand how this approximation works structurally, consider a network with ReLU activations. Each neuron j defines a hyperplane $\mathbf{w}_j^T \mathbf{x} + b_j = 0$ that splits the input space into active and inactive half-spaces. Collectively, N neurons partition the domain into multiple convex regions (polytopes) Zaslavsky 1975. Within each region, the network output is affine. Consequently, the network approximates complex topologies by constructing a global *piecewise linear* surface, whose granularity and expressiveness increase with the number of neurons.

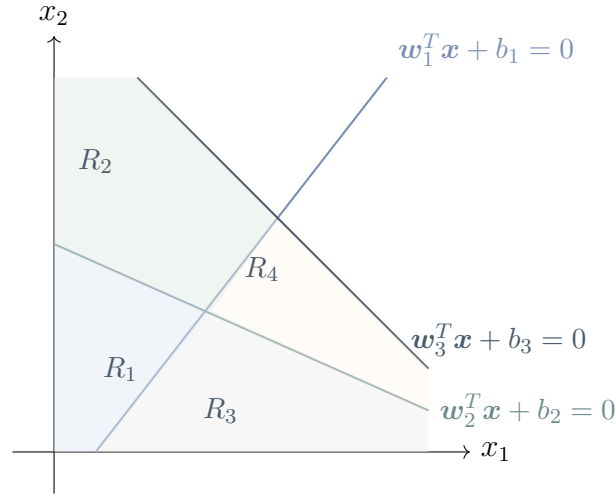


Figure 1.6: A ReLU network with 3 hidden neurons creates 3 hyperplanes that partition the 2D input distinct regions.

1.3.4 Deep vs. Shallow Networks

While the Universal Approximation Theorem establishes that shallow networks can approximate any continuous function, *depth* offers critical advantages regarding parameter efficiency and the ability to capture hierarchical structures inherent in real-world data.

Representation Efficiency and Compositionality. Deep architectures naturally leverage *compositionality*, decomposing complex tasks into simpler, hierarchical sub-problems (e.g., edges to textures to objects). Shallow networks, by contrast, must learn these mappings in a single layer, often resulting in extreme inefficiency.

Theorem 1.3 (Exponential Separation Eldan & Shamir, 2016 Eldan et al. 2016)
There exist function families that deep networks represent efficiently while shallow networks require exponentially many neurons. Concretely, a 3-layer network can compute certain functions with $\mathcal{O}(n)$ neurons, whereas any 2-layer network needs $\Omega(2^n)$ to match the accuracy.

Inductive Biases. Although the "No Free Lunch" theorem Wolpert et al. 1997 suggests no learner is universally superior, deep networks succeed because their inductive biases align with natural data (e.g., vision and language). Key biases include *hierarchy* (building complex concepts from simple ones), *smoothness*, and *locality* (exploited by CNNs via weight sharing).

Optimization and the Degradation Problem. Increasing depth theoretically improves capacity but introduces optimization challenges. Historically, naively adding layers led to the *degradation problem*, where training error increased due to vanishing gradients. Modern deep learning relies on specific remedies to stabilize training:

- **Residual connections (ResNets)** He et al. 2016 allow gradients to flow through skip connections.
- **Normalization (e.g., BatchNorm)** Ioffe et al. 2015 stabilizes internal covariate shift.
- **Specialized Initialization (e.g., He init)** He et al. 2015 prevents early gradient collapse.

Table 1.2: Comparative summary of shallow and deep architectures.

Aspect	Shallow Networks	Deep Networks
Representation	Universal but inefficient (wide)	Exponentially more efficient (deep)
Feature Learning	Direct, flat mappings	Hierarchical feature abstraction
Optimization	Easier; fewer gradients issues	Harder; requires normalization/residuals
Generalization	Suits simple tasks / small data	Dominant on complex, large-scale data
Interpretability	Features often directly interpretable	Features are abstract and distributed

1.4 Learning as Optimization

1.4.1 Empirical Risk Minimization

Given a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, the goal of learning is to find parameters $\boldsymbol{\theta}$ that minimize the discrepancy between the model's predictions $f_{\boldsymbol{\theta}}(\mathbf{x})$ and the true targets y .

We formulate this as an optimization problem known as **Empirical Risk Minimization** (ERM).

ERM minimizes the *empirical risk* $\hat{R}(\boldsymbol{\theta})$, which serves as a proxy for the inaccessible true risk (expected loss over the data distribution) Vapnik 1998:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}) \quad (1.12)$$

where $\mathcal{L}$ is a task-specific loss function.

1.4.2 Loss Functions

Regression: Mean Squared Error

For continuous targets $y \in \mathbb{R}$, the standard objective is the **Mean Squared Error** (MSE). It penalizes large deviations quadratically and provides a convex surface for linear models Bishop 2006:

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^2 \quad (1.13)$$

The gradient $\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y$ is linear, facilitating efficient backpropagation.

Classification: Cross-Entropy

For classification into K classes, the model outputs a probability distribution $\hat{\mathbf{y}}$ via the **softmax** function: $\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$. We minimize the **Cross-Entropy** loss, which maximizes the log-likelihood of the true class labels:

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad (1.14)$$

where $\mathbf{y}$ is the one-hot encoded target.

In the binary case ($K = 2$), the output layer uses a sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$, reducing the objective to **Binary Cross-Entropy**:

$$\mathcal{L}_{\text{BCE}}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad (1.15)$$

Kullback-Leibler (KL) Divergence

Closely related to cross-entropy is the **Kullback-Leibler (KL) divergence**, which measures the information loss when the predicted distribution $\hat{\mathbf{y}}$ is used to approximate the true distribution $\mathbf{y}$:

$$D_{\text{KL}}(\mathbf{y} || \hat{\mathbf{y}}) = \sum_{k=1}^K y_k \log \frac{y_k}{\hat{y}_k} = \underbrace{\sum_k y_k \log y_k}_{-H(\mathbf{y})} - \underbrace{\sum_k y_k \log \hat{y}_k}_{-\mathcal{L}_{\text{CE}}} \quad (1.16)$$

Since the entropy of the true labels $H(\mathbf{y})$ is constant with respect to the model parameters, minimizing the cross-entropy $\mathcal{L}_{\text{CE}}$ is mathematically equivalent to minimizing the KL divergence $D_{\text{KL}}(\mathbf{y}||\hat{\mathbf{y}})$.

1.5 Algorithms for Gradient-Based Optimization

Gradient-based optimization underpins the training of neural networks. The objective is to identify a parameter set $\theta \in \mathbb{R}^d$ that minimizes a scalar loss function $J(\theta)$. Given that an analytical solution for $\nabla J(\theta) = 0$ is intractable for high-dimensional non-convex surfaces, iterative numerical methods are required.

1.5.1 Gradient Descent

Standard Gradient Descent (GD) iteratively updates parameters in the direction of the negative gradient, $-\nabla J(\theta)$, representing the steepest descent. The update rule for iteration t with learning rate η is:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t) \quad (1.17)$$

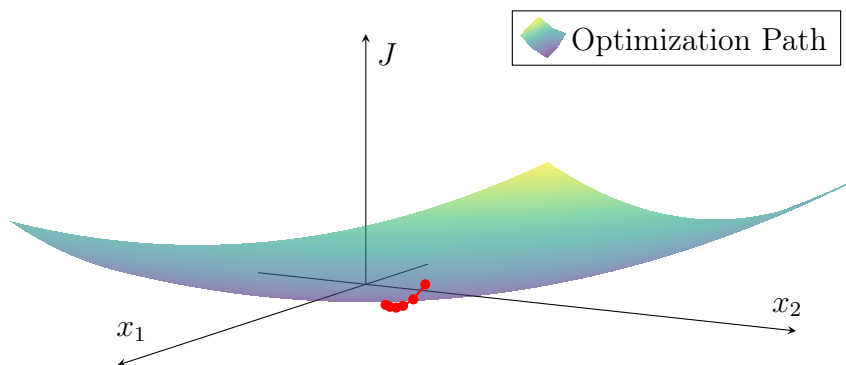


Figure 1.7: Visualization of the Gradient Descent trajectory on a convex error surface.

1.5.2 Optimization Variants

Standard GD computes the gradient over the full dataset $\mathcal{D}$, which is computationally prohibitive for large-scale learning. Modern variants address computational efficiency and convergence speed.

2

Stochastic Gradient Descent (SGD)

SGD approximates the full gradient using a *mini-batch* $\mathcal{B} \subset \mathcal{D}$ of size m . The approximate loss is:

$$J_{\mathcal{B}}(\theta) = \frac{1}{m} \sum_{(x,y) \in \mathcal{B}} L(f(x; \theta), y) \quad (1.18)$$

While $\mathbb{E}[\nabla J_{\mathcal{B}}(\theta)] = \nabla J(\theta)$, the introduced variance (noise) acts as a regularizer, aiding the escape from local minima (saddle points).

Momentum

To mitigate oscillation in high-curvature areas, Momentum accumulates a velocity vector v_t , effectively averaging past gradients Rumelhart1986:

$$v_{t+1} = \mu v_t - \eta \nabla J(\theta_t) \quad (1.19)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (1.20)$$

where $\mu \in [0, 1)$ is the momentum coefficient.

Nesterov Accelerated Gradient

Nesterov (NAG) improves momentum by computing the gradient at the *anticipated* future position:

$$v_{t+1} = \mu v_t - \eta \nabla J(\theta_t + \mu v_t) \quad (1.21)$$

This "look-ahead" capability provides a correction factor that improves convergence rates near the optimum.

Adam

Adaptive Moment Estimation (Adam) adapts the learning rate per parameter. It estimates the first (m_t) and second (v_t) raw moments of the gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1.22)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (1.23)$$

where $g_t = \nabla J(\theta_t)$. Bias-corrected estimates $\hat{m}_t$ and $\hat{v}_t$ are used for the update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (1.24)$$

Adam is currently the standard for deep learning due to its robustness to sparse gradients and stationary objectives.

1.6 The Learning Engine: Backpropagation

The *backpropagation algorithm* Rumelhart et al. 1986 enables efficient gradient computation on the directed acyclic computational graph $\mathcal{G}$ representing the neural network.

1.6.1 Forward and Backward Propagation

The network transforms inputs $\mathbf{x} = \mathbf{a}^{(0)}$ through layers $l = 1 \dots L$ via the forward pass:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}) \quad (1.25)$$

$$\mathcal{L} = \mathcal{L}(\mathbf{a}^{(L)}, \mathbf{y}) \quad (1.26)$$

To update parameters $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$, we compute gradients using the chain rule. Define the *error signal* as $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$. The backward pass computes these recursively from L down to 1:

$$\delta^{(l)} = \begin{cases} \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \odot \phi'^{(L)}(\mathbf{z}^{(L)}) & \text{if } l = L \\ [(\mathbf{W}^{(l+1)})^T \delta^{(l+1)}] \odot \phi'^{(l)}(\mathbf{z}^{(l)}) & \text{if } 1 \leq l < L \end{cases} \quad (1.27)$$

Once $\delta^{(l)}$ is obtained, the gradients for layer l are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{a}^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \quad (1.28)$$

1.7 Specialized Architectures

While Multi-layer perceptrons (MLPs) are powerful, they treat inputs as flat vectors, often ignoring spatial or sequential structures. To address this, specialized architectures were developed to capture specific data properties efficiently, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Autoencoders (AEs).

1.7.1 Convolutional Neural Networks (CNNs)

CNNs are feedforward networks optimized for grid-like data, such as images. They stack multiple modules, typically consisting of:

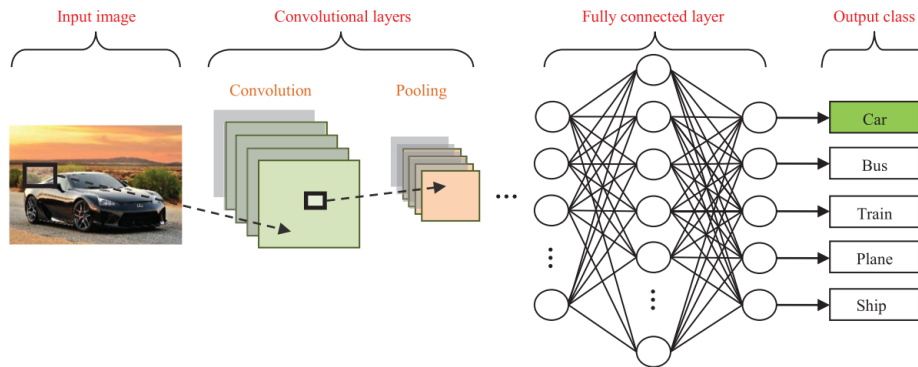


Figure 1.8: Simple CNN architecture: input, convolution + activation, pooling, and fully connected layers Waseem et al. 2017.

- **Convolutional Layers:** These act as feature extractors. A learnable filter (kernel) K slides over the input I , performing element-wise multiplication and summation to create a feature map S . The operation is defined as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

- **Pooling Layers:** These reduce the spatial dimensions (width and height) of feature maps using summary statistics (e.g., Max or Average pooling) to decrease computational cost and overfitting.

1.7.2 Recurrent Neural Networks (RNNs)

RNNs are designed for sequential data (e.g., time series, text). Unlike feedforward networks, RNNs utilize cyclic connections to maintain a *hidden state* that captures historical information. The network dynamics are described by:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

where h_t is the hidden state, x_t the input, and y_t the output at time t .

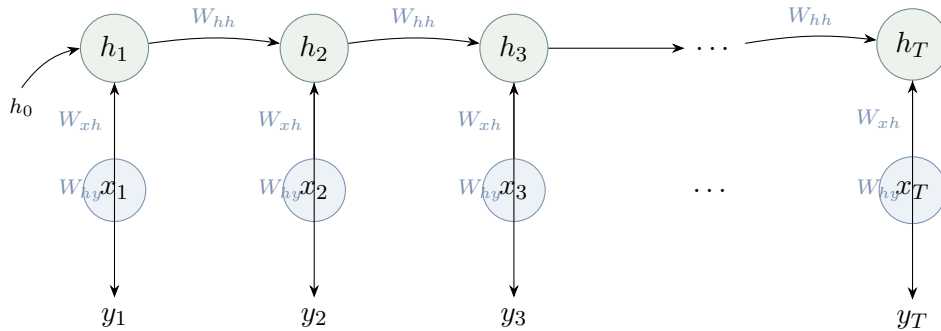


Figure 1.9: RNN unrolled in time.

1.7.3 Autoencoders (AEs)

Autoencoders are neural networks designed for unsupervised learning of efficient data codings. They act as an identity function that learns to compress and then reconstruct data. An AE consists of two main parts:

- **Encoder:** Maps the high-dimensional input x to a lower-dimensional latent representation z .
- **Decoder:** Reconstructs the output $\hat{x}$ from the latent representation z .

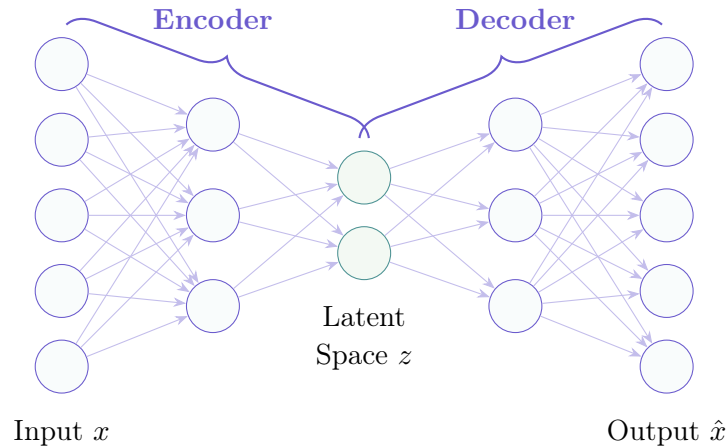


Figure 1.10: Autoencoder architecture: The input is compressed into a lower-dimensional latent space (Bottleneck) and then reconstructed.

The network is trained to minimize the reconstruction error (e.g., Mean Squared Error) between the input and the output:

$$L(x, \hat{x}) = ||x - \hat{x}||^2$$

By forcing data through a bottleneck, the network learns to capture the most salient features of the data distribution.

1.7.4 Transformer Networks

Transformer networks, introduced by Vaswani et al. 2017, revolutionized sequence modeling by relying entirely on self-attention mechanisms, dispensing with recurrence and convolution entirely. The core component of a Transformer is the *self-attention* mechanism, which allows the model to weigh the importance of different parts of the input sequence when generating each part of the output.

At the highest level, transformers for sequence-to-sequence tasks consist of two main components:

- **Encoder:** Processes the entire input sequence in parallel, building rich representations that capture the meaning and relationships of all input tokens simultaneously.
- **Decoder:** Generates the output sequence one token at a time, but with the ability to attend to all input tokens and previously generated output tokens at each step.

the architecture of a Transformer model is illustrated in Figure 1.11

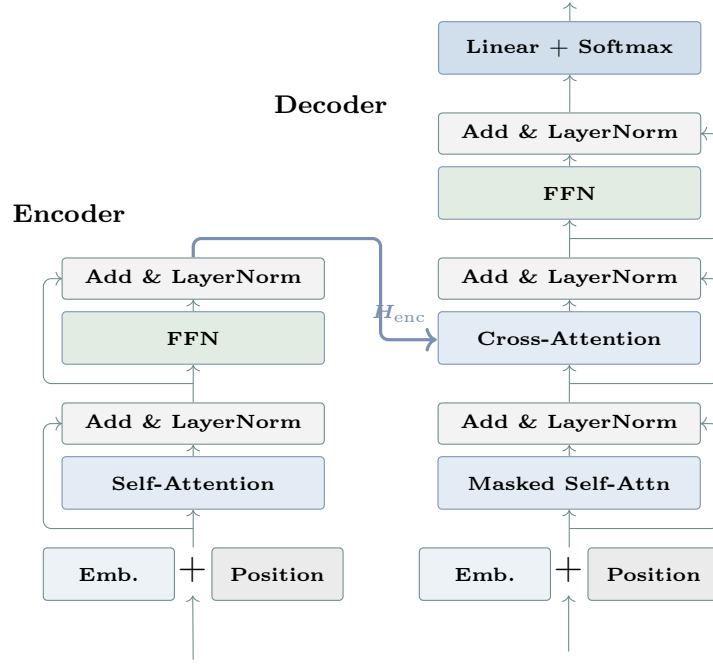


Figure 1.11: Complete Encoder-Decoder Transformer architecture. The encoder output H_{enc} (highlighted in blue) is fed to all decoder layers for cross-attention. Residual connections (curved arrows) enable stable gradient flow throughout the network.

The information flow in a Transformer can be summarized as follows:

1. Encoding Phase:

- Source tokens $\mathbf{x} = (x_1, \dots, x_n)$ are converted to embeddings and position-encoded
- The sequence flows through N encoder layers in parallel
- Each encoder layer refines the representations through self-attention and feed-forward transformations
- Final output is the encoder memory $\mathbf{H}_{enc} \in \mathbb{R}^{n \times d_{model}}$

2. Decoding Phase:

- Target tokens $\mathbf{y} = (y_1, \dots, y_t)$ are embedded and position-encoded
- Each decoder layer performs three operations in sequence:
 - (a) *Masked self-attention*: Attends to previously generated tokens (preventing information leakage from future tokens)
 - (b) *Cross-attention*: Attends to all encoder outputs $\mathbf{H}_{enc}$ to incorporate source information
 - (c) *Feed-forward transformation*: Processes the attended information

3. Output Generation:

- Final decoder output passes through a linear projection layer
- Softmax converts scores to probability distribution over vocabulary
- Model predicts the next token; this token is fed back for the next prediction step

1.7.5 Core Architectural Components: Building Blocks of the Transformer

To understand how Transformers process information, we must examine their fundamental building blocks. These components work in concert to enable the model’s powerful sequence processing capabilities.

Input Representation: Embeddings and Positional Encoding

1. **Token Embeddings:** Every token in the input vocabulary is mapped to a continuous vector representation. This learned embedding converts discrete tokens into a high-dimensional space where semantic relationships can be captured geometrically.
2. **Positional Encoding:** Unlike RNNs, which inherently process sequences in order, Transformers view input tokens as an unordered set. To inject sequence order, positional encodings are added to the token embeddings. The original Transformer uses a deterministic approach with sinusoidal functions of varying frequencies:

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \quad (1.29)$$

Each position ‘pos’ is assigned a unique encoding vector, allowing the model to learn relative positioning, as the wavelength of the sinusoids forms a geometric progression.

Stabilizing Training: Layer Normalization and Residual Connections

Training deep networks is notoriously unstable. Transformers employ two crucial techniques, applied around each sub-layer (attention and FFN), to ensure stable training and effective gradient flow.

1. **Layer Normalization (LayerNorm):** As signals propagate through the network, their magnitudes can grow or shrink uncontrollably. Layer Normalization standardizes the features for each token independently, ensuring they have a mean of zero and a variance of one. This stabilizes the activation dynamics throughout the network.
2. **Residual Connections (Skip Connections):** Borrowed from ResNets He et al. 2016, residual connections add the input of a sub-layer to its output ($\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{in}} + \text{SubLayer}(\mathbf{x}_{\text{in}})$). This creates a direct path for gradients to flow backward through the network, mitigating the vanishing gradient problem and enabling the training of much deeper models.

The Core Processing Block: The Feed-Forward Network (FFN)

After the attention mechanism aggregates information across the sequence, a position-wise Feed-Forward Network processes each token’s representation independently.

The FFN applies the same two-layer MLP to each position in the sequence:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \quad (1.30)$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are linear transformation matrices and σ is a non-linear activation function (typically ReLU or GELU).

Architectural Insight: The FFN first expands the representation to a higher-dimensional space (d_{ff} , often $4 \cdot d_{\text{model}}$) and then projects it back down. This "expand-and-contract" structure allows the model to learn more complex transformations for each token. The neurons within this FFN are a primary target for structured pruning.

The Encoder and Decoder Layers in Detail

1. **The Encoder Layer:** The encoder stack builds a powerful representation of the input sequence. Each layer consists of two main sub-layers, each followed by a residual connection and Layer Normalization:
 - (a) **Multi-Head Self-Attention:** Gathers context from the entire input sequence.
 - (b) **Feed-Forward Network (FFN):** Processes each token's resulting representation.

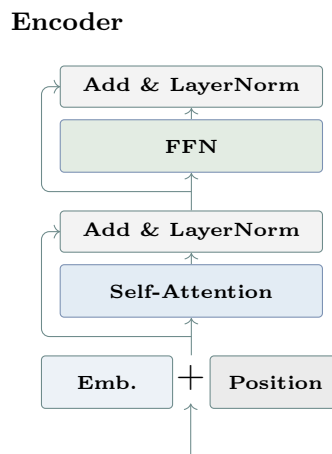


Figure 1.12: Transformer Encoder layer architecture. Residual connections (curved arrows on the left) bypass each sub-layer, ensuring stable gradient flow. Pre-layer normalization is applied before each transformation.

2. **The Decoder Layer:** The decoder generates the output sequence autoregressively. It is more complex, containing three sub-layers, each also wrapped with residual connections and Layer Normalization:
 - (a) **Masked Multi-Head Self-Attention:** Attends to previously generated tokens in the output sequence. The "mask" prevents it from looking at future tokens, preserving the autoregressive property.
 - (b) **Multi-Head Cross-Attention:** This is where the decoder "reads" the information processed by the encoder. Its Queries come from the decoder's previous sub-layer, while its Keys and Values come from the final output of the encoder stack ($\mathbf{H}_{\text{enc}}$).

- (c) **Feed-Forward Network (FFN)**: Processes the combined information from the self-attention and cross-attention steps.

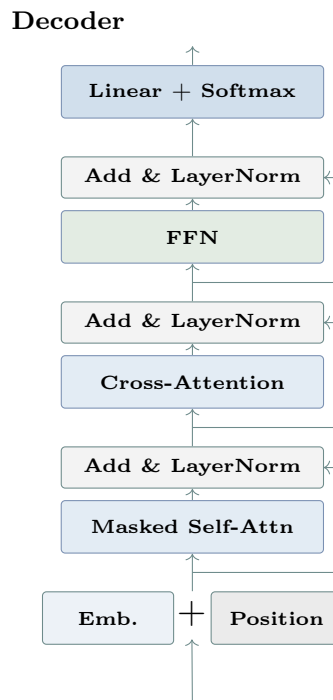


Figure 1.13: Transformer Decoder layer architecture. Note the three sub-layers: masked self-attention (to look at previously generated tokens), cross-attention (to read from encoder), and feed-forward network. Residual connections bypass each sub-layer.

Self-Attention Mechanism

Now that we understand the overall transformer architecture, we need to understand the core mechanism that makes it work: attention. This section builds the concept gradually, from simple to sophisticated.

The Core Idea: Self-attention is fundamentally a mechanism that allows each element in a sequence to gather information from all other elements. Think of it as each word in a sentence asking "Which other words are relevant to understanding me?"

The breakthrough innovation in "Attention is All You Need" Vaswani et al. 2017 was introducing three different learned projections of each input: Queries, Keys, and Values. This framework is inspired by information retrieval systems:

- **Query (q_t)**: "What information am I looking for?" - Represents the current token as a seeker of information
- **Key (k_t)**: "What information do I offer?" - Represents the current token as an advertiser of information
- **Value (v_t)**: "What is my actual content?" - The information that will be retrieved if this token is relevant

Each input token x_t is linearly projected into these three spaces using learned weight matrices:

$$q_t = W^Q x_t, \quad k_t = W^K x_t, \quad v_t = W^V x_t$$

The attention score between two tokens is calculated by the dot product of their Query and Key vectors. These scores are then normalized (via softmax) and used to create a weighted sum of the Value vectors. This process allows each token to aggregate information from the entire sequence.

Mathematically, for a sequence of length n , the self-attention output for token t is computed as:

$$\text{Attention}(q_t, K, V) = \sum_{i=1}^n \alpha_{t,i} v_i$$

where the attention weights $\alpha_{t,i}$ are given by:

$$\alpha_{t,i} = \frac{\exp(q_t \cdot k_i / \sqrt{d_k})}{\sum_{j=1}^n \exp(q_t \cdot k_j / \sqrt{d_k})}$$

Here, d_k is the dimensionality of the Key vectors, used for scaling to prevent large dot product values.

Multi-Head Attention

To enhance the model's ability to focus on different aspects of the input, Transformers employ multi-head attention. The term "multi-head" refers to having multiple attention mechanisms (or "heads") operating in parallel Vaswani et al. 2017. Each head has its own learned Query, Key, and Value transformations.

For h attention heads, each with its own weight matrices W_i^Q , W_i^K , and W_i^V for $i = 1, 2, \dots, h$:

$$\begin{aligned} q_t^i &= W_i^Q x_t \\ k_t^i &= W_i^K x_t \\ v_t^i &= W_i^V x_t \end{aligned}$$

Each head independently computes attention:

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$$

The outputs from all heads are concatenated and passed through a final linear transformation:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Where $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is a learned output projection matrix.

1.7.6 The Computational Bottleneck of Transformers

While powerful, the Transformer’s success comes at a significant cost, which is the central motivation for the compression techniques explored in this report.

- **The Quadratic Complexity of Attention:** The self-attention mechanism requires computing a similarity score between every pair of tokens in the sequence. For a sequence of length n , this results in an attention score matrix of size $n \times n$. Consequently, both the computational complexity and memory usage scale quadratically, i.e., $O(n^2)$.
- **Practical Implications:** Doubling the sequence length (e.g., from 512 to 1024) quadruples the cost of the attention computation. This makes processing very long documents prohibitively expensive and is a major bottleneck for scaling LLMs.
- **The Dichotomy of Scale:** This creates a fundamental tension: model performance scales with more parameters and data (*scaling up*), but practical deployment requires efficiency (*scaling down*).

1.8 The Bigger Picture

The transformer architecture represents a fundamental shift in how we think about sequential data processing. By replacing sequential recurrence with parallel attention, transformers have enabled a new era of scaling laws where performance correlates strongly with model size and data volume J. Kaplan et al. 2020. This architectural shift has facilitated:

- **Training on vastly larger datasets:** Parallel processing enables the ingestion of web-scale corpora, a feat previously impractical with RNNs.
- **Scaling to billions of parameters:** The "parameter race" has led to exponential growth in model size. While early models like BERT were large for their time, modern Large Language Models (LLMs) have scaled to unprecedented heights. For instance, GPT-3 utilizes 175 billion parameters Brown et al. 2020, PaLM scales to 540 billion parameters Chowdhery et al. 2023, and sparse architectures like Switch Transformers have pushed boundaries to over a trillion parameters Fedus et al. 2022.
- **Transfer learning at unprecedented scale:** These large pre-trained models generalize exceptionally well, allowing them to perform few-shot or zero-shot learning on tasks they were not explicitly trained for.
- **Applications beyond NLP:** The success of the architecture has transcended text, powering breakthroughs in computer vision (ViT), multimodal processing, and scientific discovery (e.g., AlphaFold).

While the success of transformers demonstrates that the right architectural choices, parallel computation, expressive attention mechanisms, and residual connections can unlock

capabilities previously unattainable, it comes at a significant cost. The massive increase in parameter count has led to exorbitant computational requirements, high inference latency, and substantial energy consumption Strubell et al. 2019.

This creates a dichotomy: while *scaling up* is necessary for performance, *scaling down* is often required for the realworld deployment. The sheer size of these models makes them in a way impractical for edge devices or especially low latency applications, motivating the critical need for **model compression** techniques such as quantization, pruning, and distillation to bridge the gap between research capability and production viability.

Bibliographie

- Akrout, M. et al. (2024). “Next-slot OFDM-CSI Prediction: Multi-head Self-attention or State Space Model?” In: *arXiv preprint arXiv:2405.11072*.
- Bahdanau, D., K. Cho, and Y. Bengio (2014). “Neural machine translation by jointly learning to align and translate”. In: *International Conference on Learning Representations (ICLR)*.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Brown, Tom et al. (2020). “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 1877–1901.
- Child, R. et al. (2019). “Generating long sequences with sparse transformers”. In: *arXiv preprint arXiv:1904.10509*.
- Chowdhery, Aakanksha et al. (2023). “Palm: Scaling language modeling with pathways”. In: *Journal of Machine Learning Research* 24.240, pp. 1–113.
- Cybenko, G. (1989). “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.
- Eldan, R. and O. Shamir (2016). “The power of depth for feedforward neural networks”. In: *Conference on Learning Theory*. PMLR, pp. 907–940.
- Fedus, William, Barret Zoph, and Noam Shazeer (2022). “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity”. In: *The Journal of Machine Learning Research* 23.1, pp. 5232–5270.
- Goodfellow, I., Y. Bengio, and A. Courville (2016). *Deep Learning*. MIT Press.
- Graves, A. (2013). “Generating sequences with recurrent neural networks”. In: *arXiv preprint arXiv:1308.0850*.
- Haykin, S. (1999). *Neural Networks: A Comprehensive Foundation*. 2nd. Prentice Hall.
- He, K. et al. (2015). “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification”. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026–1034.
- (2016). “Deep residual learning for image recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White (1989). “Multilayer feedforward networks are universal approximators”. In: *Neural networks* 2.5, pp. 359–366.
- Ioffe, S. and C. Szegedy (2015). “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International Conference on Machine Learning*. PMLR, pp. 448–456.
- Kaplan, Jared et al. (2020). “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361*.

- McCulloch, W. S. and W. Pitts (1943). “A logical calculus of the ideas immanent in nervous activity”. In: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133.
- Minsky, M. and S. Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Rosenblatt, F. (1958). “The Perceptron: A probabilistic model for information storage and organization in the brain”. In: *Psychological Review* 65.6, pp. 386–408.
- Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). “Learning representations by back-propagating errors”. In: *Nature* 323.6088, pp. 533–536.
- Strubell, Emma, Ananya Ganesh, and Andrew McCallum (2019). “Energy and Policy Considerations for Deep Learning in NLP”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650.
- Vapnik, V. (1998). *Statistical Learning Theory*. Wiley-Interscience.
- Vaswani, A. et al. (2017). “Attention is all you need”. In: *Advances in Neural Information Processing Systems (NIPS)*.
- Waseem, Z. and W. Zenghui (2017). “Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review”. In: *Neural Computation* 29.9, pp. 2352–2449.
- Wolpert, D. H. and W. G. Macready (1997). “No free lunch theorems for optimization”. In: *IEEE Transactions on Evolutionary Computation* 1.1, pp. 67–82.
- Zaslavsky, T. (1975). “Facing up to arrangements: face-count formulas for partitions of space by hyperplanes”. In: *Memoirs of the American Mathematical Society* 1.1, pp. 1–102.

Webographie

Raschka, Sebastian (2022). *Build a Large Language Model from Scratch: A Step-by-Step Guide*. URL: <https://sebastianraschka.com/blog/2022/llm-from-scratch.html>.

Appendices